

Trabalho A3 — ContraFácil

Plataforma de Contratação de Profissionais Autônomos

Leonardo Oliveira Morais; 1242020286
Luis Filipe da Silva Periard; 1232021702
Lucas Gabriel Lopes de Souza; 12317548
Felipe dos Santos Marques; 123115320
Matheus Tetsuo Paiva Okano; 1221121320

Lucas Goulart Silva

18/06/2026

Identificação do Projeto

O projeto ContraFácil consiste em uma aplicação back-end desenvolvida em Java, com o objetivo de apoiar o processo de contratação de profissionais autônomos.

A proposta do sistema é facilitar a conexão entre clientes que precisam contratar serviços e profissionais autônomos que desejam divulgar seu trabalho, permitindo cadastro, consulta, solicitação de contratação e avaliação dos profissionais.

O projeto foi desenvolvido como parte da disciplina Modelos, Métodos e Técnicas de Engenharia de Software, com foco na aplicação prática dos conceitos de análise de requisitos, modelagem, orientação a objetos, princípios SOLID, padrões de projeto, testes unitários e organização de código.

Definição do Problema

Atualmente, muitas pessoas enfrentam dificuldades para encontrar profissionais autônomos confiáveis para realizar serviços como eletricista, encanador, pintor, diarista, jardineiro e técnico de informática.

Da mesma forma, muitos profissionais autônomos possuem dificuldade para divulgar seus serviços, alcançar novos clientes e demonstrar a qualidade do seu trabalho de forma organizada.

O processo geralmente ocorre por meio de indicações informais, grupos de redes sociais ou aplicativos de mensagens. Esse modelo apresenta algumas limitações, como:

- dificuldade para comparar profissionais;
- ausência de uma base centralizada de prestadores de serviço;
- falta de histórico organizado de contratações;
- dificuldade para visualizar avaliações de outros clientes;

Dessa forma, o problema identificado está relacionado à falta de uma solução simples e organizada para aproximar clientes e profissionais autônomos, permitindo maior clareza na busca, contratação e avaliação dos serviços prestados.

Solução Proposta e Escopo do MVP

A solução proposta é o desenvolvimento de uma aplicação back-end em Java chamada ContraFácil, voltada para o gerenciamento básico de clientes, profissionais autônomos, solicitações de contratação e avaliações.

O objetivo do sistema é permitir que clientes encontrem profissionais de acordo com sua categoria de atuação e avaliação, além de possibilitar o registro de solicitações de contratação e avaliações dos serviços prestados.

Para fins acadêmicos, o projeto foi delimitado como um MVP, ou seja, uma versão inicial e simplificada da solução. O foco não é desenvolver uma plataforma completa, mas sim implementar uma parte relevante do sistema, demonstrando os conceitos estudados na disciplina.

Escopo implementado

O escopo do MVP contempla:

- cadastro de clientes;
- cadastro de profissionais autônomos;
- associação de profissionais a categorias;
- busca de profissionais por categoria;
- busca de profissionais por avaliação mínima;
- criação de solicitações de contratação;
- aceite ou recusa de solicitações;
- registro de avaliações;
- uso de padrões de projeto;
- aplicação dos princípios SOLID;
- testes unitários.

Fora do escopo

Não fazem parte desta entrega: interface gráfica, aplicativo mobile, autenticação real de usuários, banco de dados, Spring Boot, API REST, Swagger, pagamento online, chat entre cliente e profissional, geolocalização e integração com serviços externos.

Essas funcionalidades podem ser consideradas evoluções futuras do sistema. A decisão de mantê-las fora do escopo foi tomada para preservar a viabilidade do trabalho acadêmico e concentrar a entrega na aplicação dos conceitos de Engenharia de Software, orientação a objetos, princípios SOLID, padrões de projeto e testes unitários em Java puro.

Usuários e Atores do Sistema

O sistema possui três atores principais: Cliente, Profissional Autônomo e Administrador.

Cliente: pessoa que deseja contratar um serviço. Suas principais ações são cadastrar-se, buscar profissionais, solicitar contratação e avaliar profissionais.

Profissional Autônomo: pessoa que oferece serviços na plataforma. Suas principais ações são cadastrar-se, informar categoria, receber solicitações e aceitar ou recusar contratações.

Administrador: papel previsto para controle futuro da plataforma. Suas principais ações seriam consultar registros, acompanhar cadastros e apoiar a gestão do sistema.

Nesta versão acadêmica, o foco principal está nos fluxos de cliente e profissional autônomo. O administrador é considerado um ator previsto para evolução futura da solução.

Levantamento e Análise de Requisitos

O levantamento de requisitos foi realizado a partir da análise do problema identificado e da definição das principais necessidades dos usuários envolvidos.

A abordagem utilizada foi baseada em User Stories, permitindo representar as funcionalidades do ponto de vista dos usuários do sistema.

User Stories

US01 — Cadastro de Profissional

Como profissional autônomo,
quero me cadastrar na plataforma,
para divulgar meus serviços e ser encontrado por clientes.

Critérios de aceitação:

O nome do profissional deve ser obrigatório;

O CPF deve ser obrigatório;

A categoria profissional deve ser obrigatória;

O telefone deve ser obrigatório;

O sistema não deve permitir cadastro com dados obrigatórios ausentes.

US02 — Cadastro de Cliente

Como cliente,
quero criar uma conta,
para contratar profissionais autônomos.

Critérios de aceitação:

O nome do cliente deve ser obrigatório;

O e-mail deve ser obrigatório;

O e-mail deve ser único;

A senha deve ser obrigatória;

O sistema não deve permitir cadastro com dados obrigatórios ausentes.

US03 — Buscar Profissionais

Como cliente,
quero pesquisar profissionais por categoria,
para encontrar alguém que execute o serviço desejado.

Critérios de aceitação:

O sistema deve permitir filtro por categoria;
O sistema deve permitir filtro por avaliação mínima;
A busca deve exibir os profissionais compatíveis com os critérios informados;
O sistema deve retornar lista vazia quando nenhum profissional for encontrado.

US04 — Solicitar Contratação

Como cliente,
quero solicitar um serviço,
para contratar um profissional autônomo.

Critérios de aceitação:

O cliente deve selecionar um profissional cadastrado;
O cliente deve informar a descrição do serviço;
O sistema deve registrar a data da solicitação;
A solicitação deve possuir um status inicial.

US05 — Avaliar Profissional

Como cliente,
quero avaliar um profissional após a conclusão do serviço,
para ajudar outros usuários na escolha de profissionais.

Critérios de aceitação:

A nota deve estar entre 1 e 5;
O comentário deve ser opcional;
A avaliação deve estar vinculada a um profissional; O sistema não deve aceitar notas fora da faixa permitida.

Requisitos Funcionais

RF01 — Cadastrar cliente: o sistema deve permitir o cadastro de clientes com nome, e-mail e senha.

RF02 — Cadastrar profissional: o sistema deve permitir o cadastro de profissionais com nome, CPF, telefone e categoria.

RF03 — Pesquisar profissionais: o sistema deve permitir a busca de profissionais cadastrados.

RF04 — Filtrar profissionais por categoria: o sistema deve permitir a busca de profissionais por categoria de serviço.

RF05 — Filtrar profissionais por avaliação: o sistema deve permitir a busca considerando avaliação mínima.

RF06 — Solicitar contratação: o sistema deve permitir que um cliente solicite a contratação de um profissional.

RF07 — Aceitar ou recusar solicitação: o profissional deve poder aceitar ou recusar uma solicitação de contratação.

RF08 — Registrar avaliação: o sistema deve permitir o registro de avaliação para um profissional.

RF09 — Consultar histórico básico: o sistema deve permitir consultar registros de contratações realizadas.

Requisitos Não Funcionais

RNF01 — Linguagem Java: a aplicação deverá ser desenvolvida utilizando Java.

RNF02 — Organização em pacotes: o código deverá estar organizado em pacotes com responsabilidades separadas.

RNF03 — Orientação a Objetos: a solução deverá aplicar conceitos de classes, objetos, encapsulamento, herança, composição e abstração.

RNF04 — Princípios SOLID: o projeto deverá demonstrar aplicação dos princípios SOLID.

RNF05 — Padrões de projeto: a solução deverá utilizar padrões de projeto adequados ao contexto.

RNF06 — Testes unitários: a aplicação deverá possuir testes unitários para validar comportamentos principais.

RNF07 — Execução local: o sistema deverá ser executável localmente por meio de uma classe principal.

RNF08 — Persistência em memória: para fins acadêmicos, os dados serão armazenados em memória durante a execução da aplicação.

Regras de Negócio

As regras de negócio representam as condições e comportamentos que devem ser respeitados pela aplicação.

RN01 — Um cliente deve possuir nome, e-mail e senha.

RN02 — Um profissional deve possuir nome, CPF, telefone e categoria.

RN03 — Um profissional deve estar associado a uma categoria de serviço.

RN04 — A busca deve permitir localizar profissionais por categoria.

RN05 — A busca pode considerar uma avaliação mínima do profissional.

RN06 — Uma solicitação de contratação deve relacionar um cliente e um profissional.

RN07 — Uma solicitação deve possuir status para representar sua situação.

RN08 — O profissional pode aceitar ou recusar uma solicitação de contratação.

RN09 — Uma avaliação deve possuir nota entre 1 e 5.

RN10 — O comentário da avaliação é opcional.

RN11 — Um profissional pode possuir múltiplas avaliações.

RN12 — A avaliação média do profissional deve considerar as avaliações registradas.

Essas regras orientam a implementação das classes, serviços e testes unitários do sistema.

Arquitetura e Organização do Código

A arquitetura do ContraFácil foi organizada em pacotes com responsabilidades separadas, buscando aumentar a coesão, reduzir o acoplamento e facilitar a manutenção do código.

O projeto foi desenvolvido em Java puro, sem utilização de frameworks web ou banco de dados, pois o objetivo principal é demonstrar conceitos de Engenharia de Software, orientação a objetos, SOLID, padrões de projeto e testes unitários.

Organização dos pacotes

domain.model: contém as entidades principais do sistema, como User, Client, Provider, Category e Contract.

domain.factory: contém a fábrica responsável pela criação de usuários.

domain.repository: define abstrações e implementações de repositório utilizadas pela aplicação.

domain.service: contém serviços responsáveis pelas regras de negócio principais.

application.specification: contém as regras de filtragem reutilizáveis.

application.filter: contém o mecanismo responsável por aplicar filtros aos profissionais.

test: contém os testes unitários da aplicação.

Justificativa arquitetural

A separação em pacotes permite que cada parte do sistema tenha uma responsabilidade clara.

As entidades representam os conceitos principais do domínio, como cliente, profissional, categoria e contrato. Os serviços concentram comportamentos de negócio. Os repositórios isolam a forma de armazenamento dos dados. As especificações permitem criar filtros reutilizáveis e combináveis.

Essa organização facilita futuras evoluções do projeto, como substituição da persistência em memória por banco de dados, criação de interface gráfica ou exposição da aplicação como API.

Padrões de Projeto e SOLID

O projeto utiliza padrões de projeto para melhorar a organização, flexibilidade e manutenção do código.

Padrões de Projeto Utilizados

Factory: utilizado na classe UserFactory. Esse padrão centraliza a criação de diferentes tipos de usuários, como cliente e profissional.

Repository: utilizado nas classes ProviderRepository e InMemoryProviderRepository. Esse padrão separa as regras de negócio da forma de armazenamento dos dados.

Specification: utilizado nas classes Specification, CategorySpecification, RatingSpecification e CompositeSpecification. Esse padrão permite criar critérios de busca reutilizáveis e combináveis.

Service: utilizado nas classes ProviderService e ContractService. Esse padrão organiza comportamentos de negócio em classes específicas.

Aplicação dos Princípios SOLID

SRP — Single Responsibility Principle: cada classe possui uma responsabilidade principal, como representar entidade, criar usuário, filtrar profissionais ou gerenciar contratos.

OCP — Open/Closed Principle: novos filtros podem ser adicionados por meio de novas implementações de Specification, sem alterar o mecanismo principal de filtro.

LSP — Liskov Substitution Principle: qualquer classe que implemente Specification pode ser utilizada no mecanismo de filtro sem quebrar o comportamento esperado.

ISP — Interface Segregation Principle: as interfaces são pequenas e específicas, evitando que classes sejam obrigadas a implementar métodos desnecessários.

DIP — Dependency Inversion Principle: os serviços dependem de abstrações, como ProviderRepository, e não diretamente de uma implementação concreta.

Modelagem da Solução

A modelagem tem como objetivo representar a estrutura e o comportamento principal do sistema antes e durante a implementação.

Diagrama de Casos de Uso

O diagrama de casos de uso representa as interações entre os atores e as funcionalidades do sistema.

Atores:

- Cliente;
- Profissional Autônomo;
- Administrador.

Casos de uso principais:

- Cadastrar cliente;
- Cadastrar profissional;
- Buscar profissional;
- Filtrar profissional por categoria;
- Filtrar profissional por avaliação;
- Solicitar contratação;
- Aceitar solicitação;
- Recusar solicitação;
- Avaliar profissional;
- Consultar histórico.

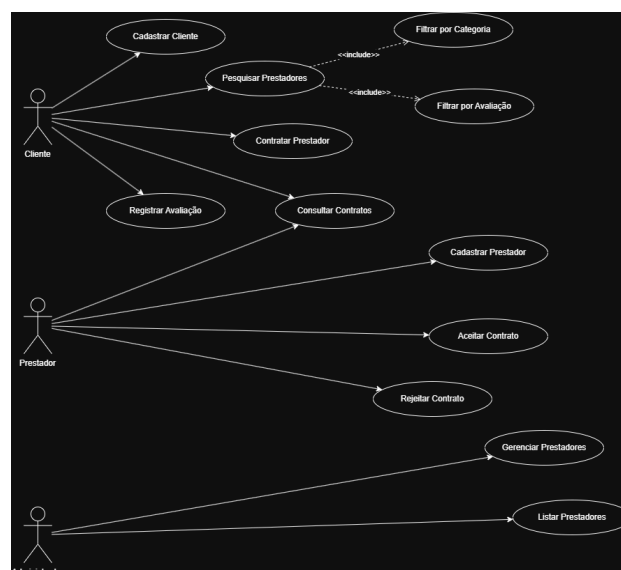
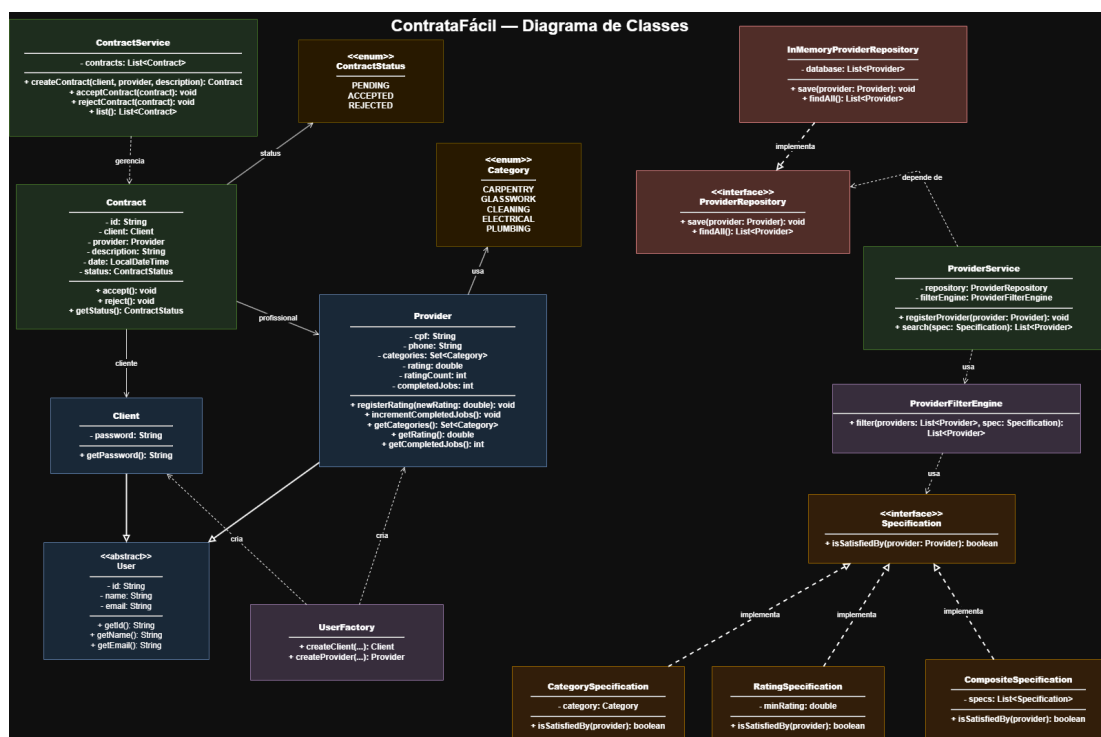


Diagrama de Classes

O diagrama de classes representa a estrutura principal da aplicação, incluindo entidades, serviços, repositórios, fábrica e especificações.

Classes principais:

- User;
- Client;
- Provider;
- Category;
- Contract;
- UserFactory;
- ProviderRepository;
- InMemoryProviderRepository;
- ProviderService;
- ContractService;
- Specification;
- CategorySpecification;
- RatingSpecification;
- CompositeSpecification;
- ProviderFilterEngine;
- ContractStatus.



Relacionamentos principais:

Client herda de User.

Provider herda de User.

Provider possui uma Category.

Contract relaciona um Client e um Provider.

ProviderService utiliza ProviderRepository.

InMemoryProviderRepository implementa ProviderRepository.

CategorySpecification, RatingSpecification e CompositeSpecification implementam Specification.

ProviderFilterEngine utiliza Specification para aplicar filtros.

UserFactory centraliza a criação de usuários.

Testes, Execução e Conclusão

Testes Unitários

Os testes unitários têm como objetivo validar o comportamento das principais regras da aplicação, garantindo que os métodos funcionem conforme esperado.

A ferramenta utilizada para os testes é o JUnit.

Classes de teste implementadas

ProviderServiceTest: valida cadastro, consulta e gerenciamento de profissionais.

ProviderFilterEngineTest: valida a aplicação de filtros sobre profissionais.

UserFactoryTest: valida a criação de clientes e profissionais.

ContractServiceTest: valida a criação e alteração de solicitações de contratação.

CategorySpecificationTest: valida o filtro por categoria.

RatingSpecificationTest: valida o filtro por avaliação mínima.

CompositeSpecificationTest: valida a combinação de filtros.

Cenários positivos

Cadastrar cliente com dados válidos.

Cadastrar profissional com dados válidos.

Buscar profissional por categoria existente.

Buscar profissional por avaliação mínima.

Criar solicitação de contratação válida.

Registrar avaliação com nota entre 1 e 5.

Cenários negativos

Tentar cadastrar profissional sem categoria.

Tentar cadastrar cliente sem e-mail.

Buscar profissionais por categoria inexistente.

Registrar avaliação com nota menor que 1.

Registrar avaliação com nota maior que 5.

Criar solicitação sem cliente ou profissional válido.

Instruções para Execução

Pré-requisitos:

Java instalado.

Maven instalado.

Git instalado.

IDE de desenvolvimento, como IntelliJ IDEA, Eclipse ou VS Code.

Clonar o repositório:

```
git clone <url-do-repositorio>  
cd contratafacil
```

Compilar o projeto:

```
mvn clean install
```

Executar os testes:

```
mvn test
```

Executar a aplicação:

```
mvn exec:java
```


Rastreabilidade entre Requisitos, Código e Testes

RF01 — Cadastrar cliente: implementação principal em Client e UserFactory; teste relacionado em UserFactoryTest.

RF02 — Cadastrar profissional: implementação principal em Provider e ProviderService; teste relacionado em ProviderServiceTest.

RF03 — Pesquisar profissionais: implementação principal em ProviderService; teste relacionado em ProviderServiceTest.

RF04 — Filtrar por categoria: implementação principal em CategorySpecification e ProviderFilterEngine; testes relacionados em CategorySpecificationTest e ProviderFilterEngineTest.

RF05 — Filtrar por avaliação: implementação principal em RatingSpecification e ProviderFilterEngine; testes relacionados em RatingSpecificationTest e ProviderFilterEngineTest.

RF06 — Solicitar contratação: implementação principal em Contract e ContractService; teste relacionado em ContractServiceTest.

RF07 — Aceitar ou recusar solicitação: implementação principal em ContractService e Contract; teste relacionado em ContractServiceTest.

RF08 — Registrar avaliação: implementação principal em Provider; teste relacionado em ProviderServiceTest.

RF09 — Consultar histórico básico: implementação principal em ContractService; teste relacionado em ContractServiceTest.

Conclusão

O desenvolvimento do ContraFácil permitiu aplicar conceitos fundamentais de Engenharia de Software em um problema real, contemplando análise de requisitos, modelagem, orientação a objetos, princípios SOLID, padrões de projeto e testes unitários.

A solução implementada demonstra uma arquitetura simples, modular e extensível, com separação de responsabilidades entre entidades, serviços, repositórios, fábrica e filtros.

Embora o sistema não tenha como objetivo entregar uma plataforma completa, ele representa uma parte relevante da solução proposta e atende ao objetivo acadêmico de aplicar, na prática, os conceitos estudados durante a disciplina.